

Sketch-Prompted VLA — evaluation of the fine-tuned checkpoint

Aaron | 19 August 2026

What I ran. I evaluated the published `prompt_conditioned_latent_action` checkpoint (`checkpoint-29999`) on the 38-scene spatial validation suite. Spatial is the one suite the checkpoint was fine-tuned on, so this is the in-distribution result; object and goal are transfer and are not reported here.

Four arms, two independent axes — prompt type (explicit / ambiguous) $\times$ sketch mode (real / blank) — at 14 rollouts per scene, so 532 rollouts per arm and 2128 in total. No rollout was skipped. The comparator is my existing $\pi_{0.5}$ -LIBERO baseline on the same 38 scenes, with the same captions, initial states and 240-step budget. The *blank* arm zeroes the sketch channel; the checkpoint never saw a zeroed sketch in training, so it measures a *degraded* sketch channel rather than its absence.

Table 1 — the four arms against the baseline

Arm	n	Success	Grasped anything	Grasped the target
Sketch-VLA, explicit + sketch	532	2.4	52.8	20.1
Sketch-VLA, explicit + blank	532	2.8	49.6	19.2
Sketch-VLA, ambiguous + sketch	532	2.6	52.6	17.7
Sketch-VLA, ambiguous + blank	532	2.6	55.5	17.7
$\pi_{0.5}$ -LIBERO, explicit	532	27.4	99.1	59.8
$\pi_{0.5}$ -LIBERO, ambiguous	532	19.4	98.1	46.1

Success is sustained success: the goal predicate held for five consecutive steps. All figures are percentages over 532 rollouts.

The sketch does nothing — and neither does the caption

Sketch against blank, on target selection: **+0.9** points under explicit captions (95% CI [-3.8, +5.7], $p = 0.70$) and **+0.0** points under ambiguous ones (CI [-4.6, +4.6], $p = 1.00$). Both are zero.

The control is the interesting half. Explicit against ambiguous captions moves the fine-tuned checkpoint by **+2.4** points (CI [-2.3, +7.1], $p = 0.31$) — also zero. On the same scenes it moves $\pi_{0.5}$ by **+13.7** points (CI [+7.8, +19.7], $p < 0.01$).

My claim is that this makes the negative result a statement about the checkpoint, not about sketch prompting. The checkpoint is a LoRA fine-tune of `pi05_libero` — the comparator itself. Sensitivity to the caption was present before the fine-tune and is absent after it. A model that has stopped using its language channel is not evidence about whether a sketch channel could work; it is evidence that this training run damaged the policy it started from. The gap on target selection is **+39.7** points ($p < 0.01$).

The mechanism is a fixed position prior, not indifference

Splitting the scenes on whether the designated object is the modal bowl instance separates a policy that reads its conditioning from one that reaches the same place every time.

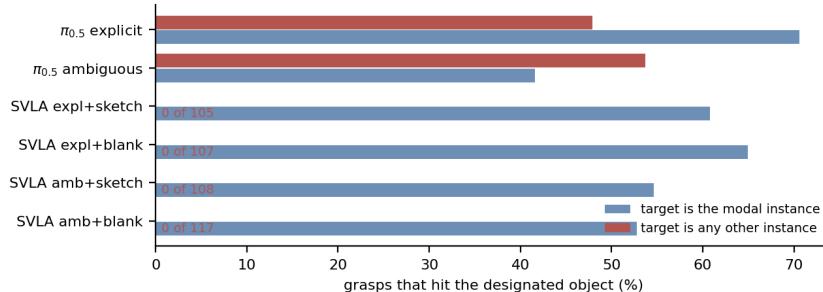


Figure 1. Of the grasps an arm actually completes, the share that landed on the designated object. Split by whether that object is the modal instance. The four fine-tuned arms have no red bar because the count is zero, not missing.

Across the four arms, on scenes whose target is not the modal instance, the policy grasped something 437 times and hit the designated object **0** times. In those same scenes it reached for the modal instance on 45.7% of its grasps. $\pi_{0.5}$ hits the target 47.9% of the time there.

Indifference to the conditioning predicts roughly chance — around 30% with two to five candidate bowls in frame. Zero out of 437 is far below chance, and a policy cannot land below chance by ignoring its inputs.

It lands there by reaching a fixed position: right when the target happens to sit there, never when it does not.

Table 2 — grasped the designated object, by tier

Tier	n	SVLA expl+sketch	SVLA amb+blank	$\pi_{0.5}$ expl	$\pi_{0.5}$ amb
Control	70	65.7	64.3	95.7	37.1
Referential	168	7.7	5.4	57.1	56.0
Directional	126	38.1	31.7	77.8	62.7
Both	168	0.0	0.0	33.9	27.4

Tier is a property of the scene: **control** one-to-one, **referential** many-to-one, **directional** one-to-many, **both** many-to-many. n is rollouts per tier in one arm.

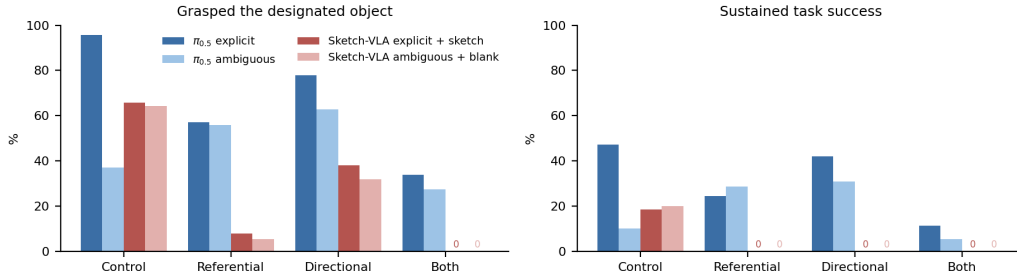


Figure 2. The same split across tiers. The fine-tuned checkpoint is competitive only on **control**, where one candidate object makes selection unnecessary, and collapses everywhere selection is required. Zeros are printed rather than left as absent bars.

What I ruled out before concluding this

- **Referent labelling.** The sketch referent derived at eval time matches the manifest ground truth 38/38, object and destination both. The zero is not a mislabelled target.
- **Image orientation.** The eval feeds unrotated frames on purpose: **sketch_libero** was rendered in the simulator’s own orientation, which is the orientation the sketches were authored in. I dump the first model-input frame of every scene and the circle lands on the intended bowl.
- **Sketch delivery.** A smoke gate refuses an empty sketch in **real** mode and a non-empty one in **blank** mode, and the mode is stamped into every row, so “blank on purpose” and “blank because the lookup missed” cannot be confused.
- **Budget parity.** Both models ran 240 steps and 48 inference calls per rollout, replanning every 5. Identical in every row of both runs.
- **Training prompts.** `prompt_from_task=True` is set for this variant, so the caption did reach the model during fine-tuning. The insensitivity is not a missing input.

Limitation

The contrasts above are unpaired. `init_state_hash` is identical across all four arms (100.0% of rows), but `init_warmstart_hash` agrees on only 7.1% — rollout 0 of each scene. The scene environment is opened once and reused for all 14 rollouts, and `set_init_state()` restores `qpos/qvel` without clearing MuJoCo’s `qacc_warmstart`, which therefore carries over from the previous rollout. With 532 rollouts per arm the unpaired test has ample power and the paired subset agrees, so I do not believe this changes the answer — but it should be fixed before the next run, by zeroing `qacc_warmstart` and `act` after the reset.

What I recommend

I would not spend GPU time on the object and goal transfer suites. The checkpoint fails in-distribution, so a transfer number would carry no information.

The likely cause sits in the fine-tuning configuration rather than in the idea: `batch_size` is 2, so 30,000 steps see 60,000 samples; warmup covers a third of the run; and `decay_steps` is 10^6 against a 30,000-step run, so the learning rate never decays. Before paying for another full run I would add a gate at every checkpoint save — one scene, two captions naming different bowls, compare the predicted action chunks. If they agree within noise the run is already dead and can be stopped in minutes.